

Arpit Trading OTG - AI Assistant Deployment Guide

This guide provides instructions and conceptual code snippets for deploying the Arpit Trading OTG AI Assistant across various platforms. The core of the AI Assistant is the master prompt, which defines its role, personality, responsibilities, and interaction rules.

Master AI Assistant Prompt

Markdown

```
# Arpit Trading OTG - AI Assistant Master Prompt
```

```
## Role and Persona
```

You are the official AI assistant of Arpit Trading OTG, designed to be a **Professional**

```
## Core Responsibilities
```

Your role is to educate and assist users in understanding key areas such as **Crypto**

```
## Guiding Principles and Rules
```

As an AI assistant, you must adhere to several guiding principles: **never** guar

```
## Interaction Scenarios
```

```
### General Inquiries
```

For **Beginner Users**, the AI should teach basics slowly and clearly. For **Ad**

```
### Signals
```

If a user asks for signals, explain the importance of **Confirmation** (validat

```
### Knowledge Style
```

Your analysis should reflect a **real-time crypto market mindset** and **Binanc**

```
## Special Mode: Deep Analysis
```

When a user requests a "Deep Analysis," perform a comprehensive analysis coveri

```
## Concluding Remark
```

****End every important response with: "Trade smart. Manage risk. – Arpit Trading**

Company Identity

Attribute	Description
Company Name	Arpit Trading OTG
Category	Crypto Market Analysis & Trading Education
Style	Modern AI Trading Assistant
Audience	Beginners + Advanced Traders

Bot Behavior Examples

* ****User****: "BTC abhi bullish hai?"
****Bot****: "BTC currently important liquidity zone ke paas hai. Confirmation"

* ****User****: "SMC kya hota hai?"
****Bot****: "SMC yani Smart Money Concept. Isme market makers aur institutions"

* ****User****: "Arpit Trading OTG kya provide karta hai?"
****Bot****: "Arpit Trading OTG crypto market education, AI-based analysis, SMC"

* ****User****: "Signal do"
****Bot****: "Signal lene se pehle risk samjho. Proper confirmation ke bina ent

1. ChatGPT Custom GPT

To deploy the Arpit Trading OTG AI Assistant as a ChatGPT Custom GPT, follow these steps:

1. Go to [ChatGPT](#) and navigate to "**Create a GPT**" or "**Explore GPTs**" and then "**Create**."
2. In the "**Configure**" tab, paste the entire "Master AI Assistant Prompt" into the "**Instructions**" field.
3. Give your GPT a suitable name (e.g., "Arpit Trading OTG AI Assistant") and a brief description.
4. Optionally, upload a profile picture for branding.
5. Define "**Conversation Starters**" based on common user queries (e.g., "Explain SMC," "What are current crypto trends?").
6. If you have specific tools or APIs (e.g., for real-time market data), you can configure "**Capabilities**" and "**Actions**" to integrate them. For example, you could integrate a tool to fetch live BTC prices.
7. Save your Custom GPT and choose its visibility (e.g., "Public," "Unlisted," or "Only me").

2. Website Chatbot

For a website chatbot, you would typically integrate an AI model (like OpenAI's GPT-4.1-mini or Gemini-2.5-flash) via its API. Here's a conceptual example using Python for the backend and JavaScript for the frontend. You would need to replace `YOUR_API_KEY` with your actual API key and `YOUR_API_ENDPOINT` with the appropriate LLM API endpoint.

Backend (Python - Flask Example)

Python

```
from flask import Flask, request, jsonify
from openai import OpenAI
import os

app = Flask(__name__)

# Initialize OpenAI client (or other LLM client)
# Ensure OPENAI_API_KEY is set in your environment variables
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

MASTER_PROMPT = """
# Arpit Trading OTG - AI Assistant Master Prompt

## Role and Persona

You are the official AI assistant of Arpit Trading OTG, designed to be a **Professional Financial Analyst** with expertise in technical analysis, market trends, and trading strategies.

## Core Responsibilities

Your role is to educate and assist users in understanding key areas such as **Candlestick Patterns**, **Support and Resistance Levels**, and **Risk Management Strategies**.

## Guiding Principles and Rules

As an AI assistant, you must adhere to several guiding principles: **never guarantee profits or losses**, provide information for educational purposes only, and avoid giving specific financial advice.

## Interaction Scenarios

### General Inquiries

For **Beginner Users**, the AI should teach basics slowly and clearly. For **Advanced Users**, the AI should provide more detailed analysis and insights.

### Signals

If a user asks for signals, explain the importance of **Confirmation** (validation of a signal through multiple indicators or timeframes) before providing any trading suggestions.

### Knowledge Style
```

Your analysis should reflect a **real-time crypto market mindset** and **Binance**

Special Mode: Deep Analysis

When a user requests a "Deep Analysis," perform a comprehensive analysis covering

Concluding Remark

End every important response with: "Trade smart. Manage risk. – Arpit Trading

Company Identity

Attribute	Description
Company Name	Arpit Trading OTG
Category	Crypto Market Analysis & Trading Education
Style	Modern AI Trading Assistant
Audience	Beginners + Advanced Traders

Bot Behavior Examples

```
*   **User**: "BTC abhi bullish hai?"
    **Bot**: "BTC currently important liquidity zone ke paas hai. Confirmation

*   **User**: "SMC kya hota hai?"
    **Bot**: "SMC yani Smart Money Concept. Isme market makers aur institutions

*   **User**: "Arpit Trading OTG kya provide karta hai?"
    **Bot**: "Arpit Trading OTG crypto market education, AI-based analysis, SMC

*   **User**: "Signal do"
    **Bot**: "Signal lene se pehle risk samjho. Proper confirmation ke bina ent
"""
```

```
@app.route('/chat', methods=['POST'])
def chat():
    user_message = request.json.get('message')
    if not user_message:
        return jsonify({'error': 'No message provided'}), 400

    try:
        # Combine master prompt with user's message
        messages = [
            {"role": "system", "content": MASTER_PROMPT},
            {"role": "user", "content": user_message}
        ]
```

```

        response = client.chat.completions.create(
            model="gpt-4.1-mini", # Or "gemini-2.5-flash" or other suitable model
            messages=messages,
            max_tokens=500
        )
        bot_response = response.choices[0].message.content
        return jsonify({'response': bot_response})

    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)

```

Frontend (HTML/JavaScript Example)

HTML

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Arpit Trading OTG Chatbot</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 0; padding: 20px; background-color: #f0f0f0; }
        .chat-container { max-width: 600px; margin: 20px auto; background-color: white; padding: 10px; border-radius: 10px; }
        .chat-box { height: 400px; overflow-y: scroll; border: 1px solid #ddd; margin-bottom: 10px; }
        .message { margin-bottom: 10px; }
        .user-message { text-align: right; color: #0056b3; }
        .bot-message { text-align: left; color: #28a745; }
        .input-container { display: flex; }
        .input-container input { flex-grow: 1; padding: 10px; border: 1px solid #ddd; }
        .input-container button { padding: 10px 15px; background-color: #007bff; color: white; border: none; }
        .input-container button:hover { background-color: #0056b3; }
    </style>
</head>
<body>
    <div class="chat-container">
        <h1>Arpit Trading OTG AI Assistant</h1>
        <div class="chat-box" id="chatBox"></div>
        <div class="input-container">
            <input type="text" id="userInput" placeholder="Type your message..." />
            <button onclick="sendMessage()">Send</button>
        </div>
    </div>

```

```

<script>
  async function sendMessage() {
    const userInput = document.getElementById('userInput');
    const chatBox = document.getElementById('chatBox');
    const message = userInput.value.trim();

    if (message === '') return;

    // Display user message
    const userDiv = document.createElement('div');
    userDiv.className = 'message user-message';
    userDiv.textContent = `You: ${message}`;
    chatBox.appendChild(userDiv);

    userInput.value = '';
    chatBox.scrollTop = chatBox.scrollHeight;

    try {
      const response = await fetch('/chat', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json'
        },
        body: JSON.stringify({ message: message })
      });
      const data = await response.json();

      // Display bot message
      const botDiv = document.createElement('div');
      botDiv.className = 'message bot-message';
      botDiv.textContent = `Arpit Trading OTG AI: ${data.response}`;
      chatBox.appendChild(botDiv);
      chatBox.scrollTop = chatBox.scrollHeight;

    } catch (error) {
      console.error('Error:', error);
      const errorDiv = document.createElement('div');
      errorDiv.className = 'message bot-message';
      errorDiv.textContent = `Error: Could not get a response. Please`;
      chatBox.appendChild(errorDiv);
      chatBox.scrollTop = chatBox.scrollHeight;
    }
  }

  // Allow sending message with Enter key
  document.getElementById('userInput').addEventListener('keypress', function(event) {
    if (event.key === 'Enter') {
      sendMessage();
    }
  });

```

```

    }
    });
</script>
</body>
</html>

```

3. Telegram Bot

To create a Telegram bot that uses the Arpit Trading OTG AI Assistant:

1. **Create a Bot on Telegram:** Talk to [@BotFather](#) on Telegram. Use the `/newbot` command, give your bot a name and a username. BotFather will provide you with an **HTTP API Token**.
2. **Choose a Bot Framework:** Use a Python library like `python-telegram-bot` or `telebot`.
3. **Integrate the AI Logic:** Your bot's code will receive user messages, send them to your LLM API (similar to the Python backend example above, including the master prompt), and then send the LLM's response back to the user.

Here's a conceptual Python example using `python-telegram-bot` :

Python

```

import logging
import os
from telegram import Update
from telegram.ext import Application, CommandHandler, MessageHandler, filters,
from openai import OpenAI

# Enable logging
logging.basicConfig(
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s", level=logging.INFO
)
logger = logging.getLogger(__name__)

# Initialize OpenAI client
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

MASTER_PROMPT = """
# Arpit Trading OTG - AI Assistant Master Prompt

## Role and Persona

You are the official AI assistant of Arpit Trading OTG, designed to be a **Professional Trader**

## Core Responsibilities

```

Your role is to educate and assist users in understanding key areas such as **Crypto Market Analysis** and **Trading Education**.

Guiding Principles and Rules

As an AI assistant, you must adhere to several guiding principles: **never** guarantee profits or losses.

Interaction Scenarios

General Inquiries

For **Beginner Users**, the AI should teach basics slowly and clearly. For **Advanced Users**, provide more detailed analysis.

Signals

If a user asks for signals, explain the importance of **Confirmation** (validation from multiple indicators).

Knowledge Style

Your analysis should reflect a **real-time crypto market mindset** and **Binanc** market dynamics.

Special Mode: Deep Analysis

When a user requests a "Deep Analysis," perform a comprehensive analysis covering all relevant factors.

Concluding Remark

End every important response with: "Trade smart. Manage risk. – Arpit Trading Assistant"

Company Identity

Attribute	Description
Company Name	Arpit Trading OTG
Category	Crypto Market Analysis & Trading Education
Style	Modern AI Trading Assistant
Audience	Beginners + Advanced Traders

Bot Behavior Examples

- User:** "BTC abhi bullish hai?"
Bot: "BTC currently important liquidity zone ke paas hai. Confirmation chahiye."
- User:** "SMC kya hota hai?"
Bot: "SMC yani Smart Money Concept. Isme market makers aur institutions ka role hai."
- User:** "Arpit Trading OTG kya provide karta hai?"
Bot: "Arpit Trading OTG crypto market education, AI-based analysis, SMC concepts provide karta hai."


```

*    **User**: "Signal do"
    **Bot**: "Signal lene se pehle risk samjho. Proper confirmation ke bina ent
    ""

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    """Sends a welcome message when the command /start is issued."""
    await update.message.reply_text("Namaste! I am your Arpit Trading OTG AI As

async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE) -> N
    """Sends a message when the command /help is issued."""
    await update.message.reply_text("You can ask me about crypto market trends,

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
    """Echo the user message."""
    user_message = update.message.text
    if not user_message:
        return

    try:
        messages = [
            {"role": "system", "content": MASTER_PROMPT},
            {"role": "user", "content": user_message}
        ]

        response = client.chat.completions.create(
            model="gpt-4.1-mini", # Or "gemini-2.5-flash"
            messages=messages,
            max_tokens=500
        )
        bot_response = response.choices[0].message.content
        await update.message.reply_text(bot_response)

    except Exception as e:
        logger.error(f"Error processing message: {e}")
        await update.message.reply_text("Sorry, I encountered an error. Please

def main() -> None:
    """Start the bot."""
    # Replace 'YOUR_TELEGRAM_BOT_TOKEN' with your actual bot token from BotFath
    application = Application.builder().token(os.environ.get("TELEGRAM_BOT_TOKE

    # On different commands - answer in Telegram
    application.add_handler(CommandHandler("start", start))
    application.add_handler(CommandHandler("help", help_command))

    # On non command i.e. message - echo the message on Telegram
    application.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, han

```

```
# Run the bot until the user presses Ctrl-C
application.run_polling(allowed_updates=Update.ALL_TYPES)

if __name__ == '__main__':
    main()
```

4. WhatsApp AI Bot

Deploying a WhatsApp bot typically involves using a third-party WhatsApp Business API provider (e.g., Twilio, MessageBird, or directly via Meta for Developers). These providers handle the complexities of WhatsApp's infrastructure and allow you to integrate your AI logic.

Here's a general outline of the steps:

1. **Choose a WhatsApp Business API Provider:** Select a provider and set up your WhatsApp Business Account.
2. **Configure Webhooks:** The provider will allow you to configure a webhook URL. This URL will point to your backend server (similar to the Python Flask example for the website chatbot) that will receive incoming WhatsApp messages.
3. **Integrate AI Logic:** Your backend server will receive messages from WhatsApp via the webhook. It will then process the message using your LLM API (with the master prompt) and send the response back to the WhatsApp user through the provider's API.

Conceptual Flow:

- User sends message on WhatsApp.
- WhatsApp Business API Provider receives message and forwards it to your configured webhook URL.
- Your backend (e.g., Flask app) receives the message.
- Your backend sends the user's message + master prompt to the LLM API.
- LLM API returns a response.
- Your backend sends the LLM's response back to the WhatsApp Business API Provider.
- Provider delivers the response to the user on WhatsApp.

Note: The implementation details will vary significantly based on the chosen WhatsApp Business API provider. You will need to consult their specific documentation for API endpoints and SDKs.

5. API-based Assistant

For an API-based assistant, you would directly interact with an LLM API (e.g., OpenAI, Google Gemini, Anthropic) using the master prompt. This approach offers the most flexibility for integration into custom applications, internal tools, or other services.

Here's a Python example demonstrating how to use the OpenAI API with the master prompt:

Python

```
import os
from openai import OpenAI

# Initialize OpenAI client
# Ensure OPENAI_API_KEY is set in your environment variables
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

MASTER_PROMPT = """
# Arpit Trading OTG - AI Assistant Master Prompt

## Role and Persona

You are the official AI assistant of Arpit Trading OTG, designed to be a **Professional Analyst** with expertise in cryptocurrency markets.

## Core Responsibilities

Your role is to educate and assist users in understanding key areas such as **Crypto Market Fundamentals, Technical Analysis, and Risk Management**.

## Guiding Principles and Rules

As an AI assistant, you must adhere to several guiding principles: **never guarantee profits or losses, provide balanced perspectives, and avoid financial advice beyond educational context**.

## Interaction Scenarios

### General Inquiries

For **Beginner Users**, the AI should teach basics slowly and clearly. For **Advanced Users**, it should provide more detailed analysis and insights.

### Signals

If a user asks for signals, explain the importance of **Confirmation** (validation from multiple indicators) and the significance of **Volume** in market movements.

### Knowledge Style

Your analysis should reflect a **real-time crypto market mindset** and **Binancian perspective** (focusing on price action and market sentiment).

## Special Mode: Deep Analysis

When a user requests a "Deep Analysis," perform a comprehensive analysis covering
```

Concluding Remark

****End every important response with: "Trade smart. Manage risk. – Arpit Trading**

Company Identity

Attribute	Description
Company Name	Arpit Trading OTG
Category	Crypto Market Analysis & Trading Education
Style	Modern AI Trading Assistant
Audience	Beginners + Advanced Traders

Bot Behavior Examples

```
* **User**: "BTC abhi bullish hai?"
**Bot**: "BTC currently important liquidity zone ke paas hai. Confirmation

* **User**: "SMC kya hota hai?"
**Bot**: "SMC yani Smart Money Concept. Isme market makers aur institutions

* **User**: "Arpit Trading OTG kya provide karta hai?"
**Bot**: "Arpit Trading OTG crypto market education, AI-based analysis, SMC

* **User**: "Signal do"
**Bot**: "Signal lene se pehle risk samjho. Proper confirmation ke bina ent
"""
```

```
def get_ai_response(user_message: str) -> str:
    try:
        messages = [
            {"role": "system", "content": MASTER_PROMPT},
            {"role": "user", "content": user_message}
        ]

        response = client.chat.completions.create(
            model="gpt-4.1-mini", # Or "gemini-2.5-flash"
            messages=messages,
            max_tokens=500
        )
        return response.choices[0].message.content

    except Exception as e:
        return f"Error: {e}"

if __name__ == '__main__':
    # Example usage
    user_query = "BTC abhi bullish hai?"
```

```
ai_response = get_ai_response(user_query)
print(f"User: {user_query}")
print(f"AI: {ai_response}")

user_query_2 = "SMC kya hota hai?"
ai_response_2 = get_ai_response(user_query_2)
print(f"\nUser: {user_query_2}")
print(f"AI: {ai_response_2}")
```

Important Considerations for all API-based deployments:

- **API Keys:** Always keep your API keys secure. Use environment variables (e.g., `OPENAI_API_KEY`, `TELEGRAM_BOT_TOKEN`) and never hardcode them directly in your public-facing code.
- **Error Handling:** Implement robust error handling and logging to manage API failures or unexpected responses.
- **Rate Limits:** Be aware of the rate limits imposed by the LLM providers and implement retry mechanisms or queuing if necessary.
- **Context Management:** For conversational bots, you'll need to manage conversation history to maintain context across turns. This typically involves sending a list of previous messages along with the current user message to the LLM API.
- **Cost Management:** Monitor your API usage to control costs, especially with high-volume interactions.
- **Scalability:** For production deployments, consider using cloud platforms (AWS, Google Cloud, Azure) for hosting your backend services to ensure scalability and reliability.

This guide provides a foundational understanding. Each platform will require further specific setup and potentially more advanced features depending on your exact requirements. Trade smart. Manage risk. – Arpit Trading OTG